Facade Pattern

The Facade Pattern is a structural design pattern that provides a streamlined interface to a complex system of classes, a library, or a framework. It defines a higher-level interface that facilitates the subsystem's use but does not prevent you from using the complex system directly.

Facade Pattern can be explained by using the Computer System example. When you switch on your computer system, some intricate steps are involved, for example, loading the boot sector from the hard drive, loading the OS into memory, and initializing the CPU.

Without using Facade Pattern, you need to perform all these steps by yourself. But, the Facade pattern encapsulates all these steps behind a start button. Press the start button, and all the system components are loaded in the memory and ready to use.

Real-World Example

Consider yourself using a home theater system that includes a game console, TV, DVD player, and sound system. There are several steps involved in turning on all of these components, configuring their input and output settings, and then actually beginning to play a game or watch a movie.

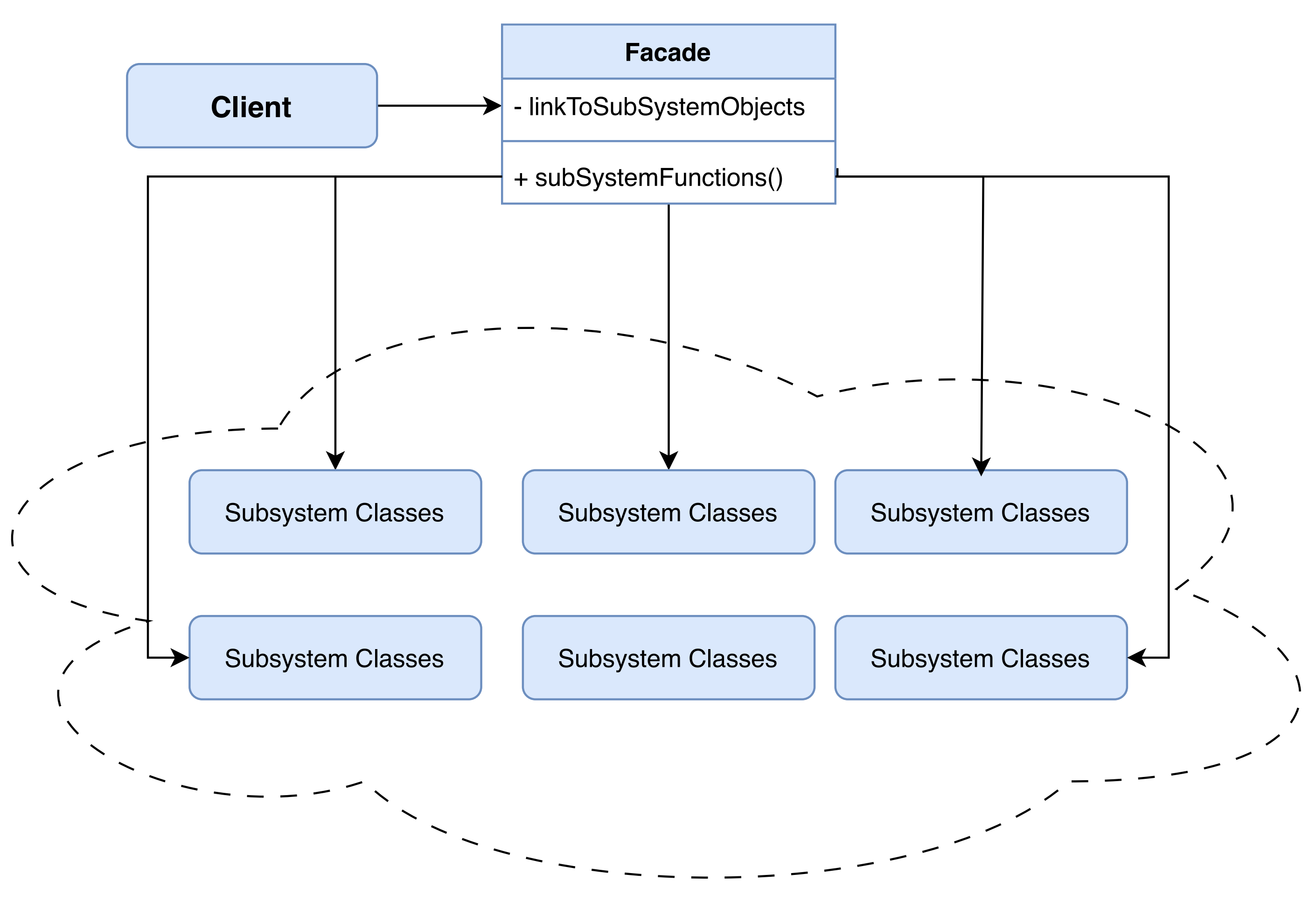
Facade Pattern - Real-World Example

As this image illustrates, with just one button press, the Facade Pattern can perform all these actions, much like a "smart home" interface or a single remote control. It simplifies the process, but all of the individual system complexities remain behind the scenes for those who want to manage them directly.

Structure of Facade Pattern

The Facade Pattern has three main components:

Facade: Gives clients access to basic methods that transfer the call to the methods of existing system classes. **Subsystem Classes**: The classes with which the facade communicates inside the complex system. **Client**: Uses the facade rather than making direct calls to the subsystem objects.



Facade Pattern - Class Diagram

Implementation of Facade Pattern

For implementation, let's look at the implementation of computer system example. There are multiple classes for this example:

- `Computer` - **Facade Class**:

  This class acts as a facade for the intricate processes required to boot the computer. It offers a method that encapsulates the actions required to boot the computer, like `startComputer()`.

- **Subsystem Classes**: `Memory`, `Hard Drive`, `Operating System`, **and** `CPU`

  These classes represent various computer components. Every class has unique, in-depth functionalities. For instance, the `Operating System` has `loadKernel()`, the `Hard Drive` has `readBootSector()`, the `CPU` has `initialize()`, and `Memory` has `load()`.

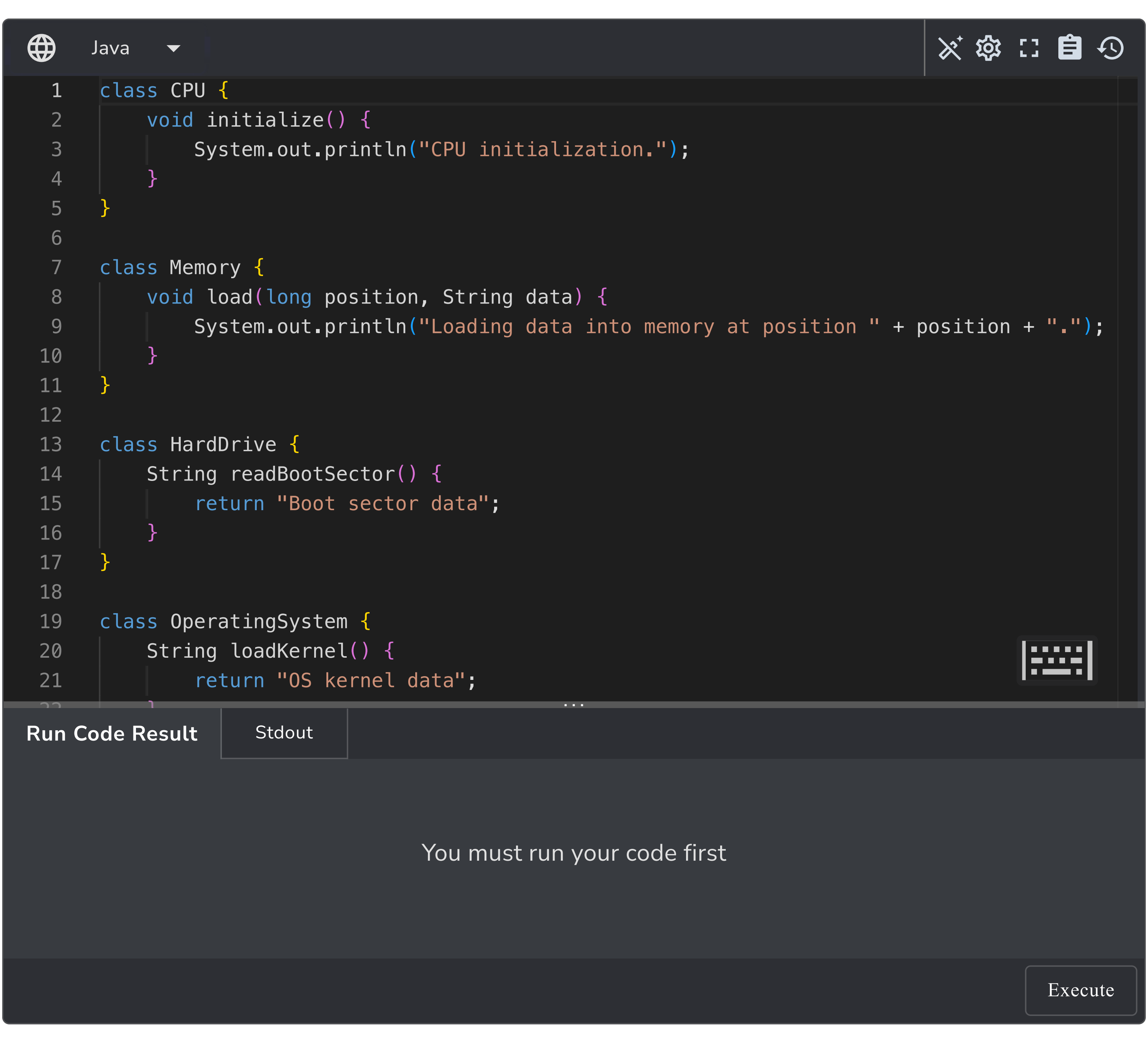
- **Client**:

  To turn on the computer, the client interacts with the computer facade. It is not required to be aware of the minute details within the subsystems.

Pseudocode

```
CLASS CPU
    METHOD initialize()
        // Initialize CPU

CLASS Memory
    METHOD load(position, data)
        // Load data into memory at a specific position

CLASS HardDrive
    METHOD readBootSector()
        // Read data from boot sector

CLASS OperatingSystem
    METHOD loadKernel()
        // Load OS kernel into memory

CLASS Computer (Facade)
    PRIVATE cpu: CPU
    PRIVATE memory: Memory
    PRIVATE hardDrive: HardDrive
    PRIVATE operatingSystem: OperatingSystem

    CONSTRUCTOR()
        cpu = new CPU()
        memory = new Memory()
        hardDrive = new HardDrive()
        operatingSystem = new OperatingSystem()

    METHOD startComputer()
        bootSector = hardDrive.readBootSector()
        osData = operatingSystem.loadKernel()
        memory.load(BOOT_POSITION, bootSector)
        memory.load(OS_POSITION, osData)
        cpu.initialize()

// Client code
computer = new Computer()
computer.startComputer()
```

In this example, the `Computer` class (Facade) simplifies the computer startup procedure. The client is not required to interact with or comprehend the underlying subsystems' complexity. This encapsulation preserves the full functionality of the computer system while making the client code cleaner and simpler to maintain.



```java
class CPU {
    void initialize() {
        System.out.println("CPU initialization.");
    }
}

class Memory {
    void load(long position, String data) {
        System.out.println("Loading data into memory at position " + position + ".");
    }
}

class HardDrive {
    String readBootSector() {
        return "Boot sector data";
    }
}

class OperatingSystem {
    String loadKernel() {
        return "OS kernel data";
```

You must run your code first

Applications of Facade Pattern

The Facade Pattern is especially useful in the following scenarios:

- **Simplifying Complex Systems**: The Facade Pattern provides a straightforward, high-level interface for working with complex subsystems made up of multiple classes or modules. This lowers the complexity that clients have to control and facilitates easier subsystem use.
- **Reducing Dependencies**: The Facade Pattern helps reduce dependencies in systems where clients rely largely on implementation classes. Clients can avoid the intricate details of the subsystem's implementation by interacting with a facade rather than the subsystem directly.
- **Subsystem Level Interface**: The Facade Pattern helps define a clear entry point for each subsystem in systems organized into layers or subsystems. It makes the system architecture more organized and clear by enabling communication between all subsystems via a single interface.
- **External library or API encapsulation**: The Facade Pattern can be used to encapsulate external components when integrating them. This makes it easier to use them in your application and makes it possible to replace the libraries at a later date with little to no effect on the remainder of it.

The Facade Pattern deals with providing a unified, simplified interface to a group of interfaces in a subsystem, thereby reducing system complexity and improving usability.

Pros and Cons

Pros	Cons
Simplicity: Provides a simple interface to a complex system.	**Limited Access**: Can limit the features and flexibility that "power users" might need.
Reduced Dependencies: Minimizes the communication and dependencies between systems.	**Over-Simplification**: May oversimplify the system, hiding features that advanced users might require.
Ease of Use: Makes the subsystem easier to use, understand, and test.	**Changes in Subsystem**: Changes inside the subsystem may lead to changes in the facade.
Isolation: Shields clients from subsystem components, promoting loose coupling.	

The Facade Pattern is a super cool design trick that cleans up your interactions with complex systems. Think of it as the friendly face that simplifies all the nitty-gritty details behind the scenes. Instead of wrestling with a bunch of complicated classes, you get to work with just one neat, tidy interface.

← Previous
Decorator Pattern

Next →
Flyweight Pattern

☐ Mark as Completed